Maze Escape: Human vs Monster AI Survival Game

Principles of Artificial Intelligence Project Report

Course: Principles of Artificial Intelligence
Project: Maze Escape: Human vs Monster AI Survival Game

June 25, 2026

Contents

Abstract

To explore innovative applications of AI search algorithms and **machine learning**—and to evaluate the performance of different algorithms in specific contexts—we developed a game called "Maze Escape." The game simulates a scenario where a human is pursued by a monster across a 30×30 toroidal grid map (featuring wrap-around edges); the monster's behavior is driven by search algorithms and models trained via machine learning. The system compares five AI search strategies: **A* pathfinding**, **Greedy Search**, **Breadth-First Search (BFS)**, **Minimax**, and a **simulated annealing algorithm**. The mazes are generated using a **Genetic Algorithm (GA)**, which evolves the map over multiple generations to optimize layout, obstacle distribution, path diversity, and branching structures. Additionally, we incorporated machine learning to train a model capable of predicting the human's position three steps ahead, thereby challenging the player's escape efforts.

After verifying the game's logic and operational stability, we utilized the game as a platform to successfully evaluate the performance of these algorithmic methods and the machine learning model. Our findings indicate that the A* algorithm generates the most direct pursuit path; the Greedy algorithm offers rapid performance at a low computational cost; the simulation algorithm introduces controlled randomness, making the monster's movements harder to predict; and the Genetic Algorithm consistently produces connected, navigable maps that maintain high game quality.

1 Introduction

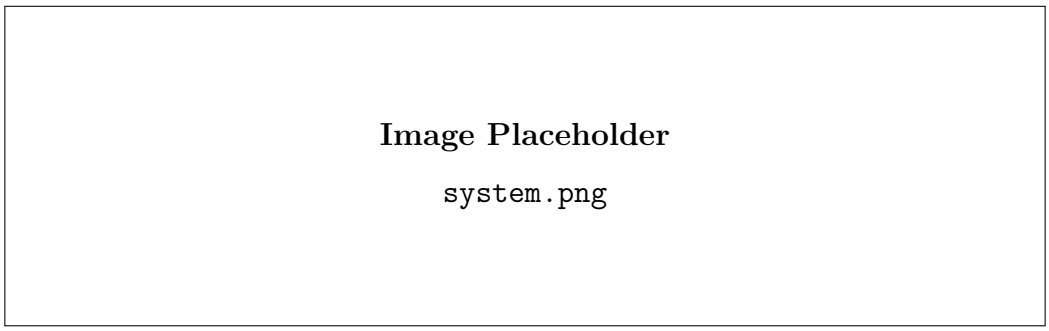


Figure 1: Game System

In the course "Principles of Artificial Intelligence," we studied a variety of search, planning, and optimization algorithms. Based on these core methodologies, our Maze Escape project extends static, single-agent pathfinding into dynamic, multi-dimensional adversarial scenarios. To achieve this transition, we introduce randomly generated maze maps based on genetic algorithms, expand the dimensional space, and a two-way tracking mechanism enhanced by predictive machine learning has been realized.

In order to comprehensively evaluate these algorithms, we have designed two complementary game modes: escape mode and chase mode. In the chase mode, the actions of human characters are driven by evasion algorithms based on Breadth-First Search (BFS). For the core task of predators pursuing prey, the project adopts four distinct intelligent strategies and combined with machine learning-driven interception mechanism:

- **A* Search Algorithm:** In single-monster mode, the algorithm calculates the shortest path to the player. In dual-monster mode, monsters are divided into two types: direct pursuers and interceptors. The interceptors predict the player's next move and block potential escape routes.
- **Greedy Algorithm:** It calculates a distance field centered at the player's current position, causing monsters to always move toward the nearest neighboring node. This pursuit method is highly efficient and requires low computational resources.
- **Minimax Decision Tree:** This strategy treats the pursuit process as a two-player zero-sum game. The algorithm combines α-β pruning with a transposition table to predict subsequent moves. The monster aims to minimize the player's safety score during the game, while simultaneously simulating the player's efforts to maximize it, thereby enabling efficient pursuit from both perspectives.
- **Simulated Annealing Algorithm:** The algorithm first generates an initial movement path and then randomly adjusts it through repeated iterations. Following the cooling schedule, the system accepts suboptimal moves with a certain probability, making the monster's actions more stochastic and harder to predict.
- **Machine Learning:** Based on spatial characteristics (the relative position of humans relative to monsters, local wall obstacles and the direction of movement of the first two steps), we have trained a random forest model to predict the next move of human players, and predict the position of the next three steps by rolling, so as to achieve effective blockade.

Our game environment is a 30×30 circular grid map, where the specific layout of the maze is generated by a modified genetic algorithm (GA). This algorithm maintains a population of candidate maps and evolves them over more than 20 generations to ensure the highest quality final map. The fitness function of the genetic algorithm considers several key factors, including good ground connectivity, approximately 28% obstacle quantity, the distribution of three-way and four-way intersections, and a breadth-first search (BFS) path length of approximately 20 steps between randomly selected locations. We also employed tournament selection, row and column crossover, and bit-flip mutation methods to effectively help the algorithm generate maps with high accessibility, spatial diversity, and appropriate difficulty.

The following sections of this report will detail the design and implementation of each component, quantitatively analyze the algorithmic characteristics of each AI strategy and its impact on gameplay, and reflect on their respective advantages and limitations observed during development.

2 Methodology

2.1 Genetic Algorithm for Map Generation

The map generation module uses a genetic algorithm to automatically generate the game map. The aim of this part is not just to place walls randomly, but to create a 30×30 map that is connected, playable, and suitable for the chase mechanism in this game. In this project, the map structure directly affects the game balance. If the map is too open,

monsters may chase the human too easily. If there are too many walls, the movement of both the human and monsters may become restricted, and they may even be separated into unreachable areas. For this reason, the map generation process needs to balance randomness and playability.

The map is represented as a 30×30 two-dimensional grid. Each cell is stored as a binary value. The value `0` represents walkable ground, while the value `1` represents a wall or obstacle. The human and monsters can only move on walkable cells. The map also uses a wrap-around boundary mechanism, which means that when a character moves out of one side of the map, it will appear from the opposite side. Because of this design, modulus operation is used when calculating neighbouring cells, and Breadth-First Search (BFS) can be applied on this wrap-around map structure for connectivity and path distance checking.

In the genetic algorithm, each map is regarded as a chromosome, while a group of maps forms a population. The initial population is generated randomly, where each cell is assigned as either walkable ground or obstacle according to a fixed wall probability. In this implementation, the target wall ratio is set to around 0.28, which means that about 28% of the map is expected to be walls. This value is used to avoid maps that are either too empty or too crowded.

After the initial maps are generated, each candidate map is evaluated by a fitness function. The fitness function mainly considers four factors: reachability, wall ratio, junction complexity, and path distance. The final fitness score can be expressed as:

$$\text{Fitness} = S_{\text{reachability}} + S_{\text{wall ratio}} + S_{\text{junction}} + S_{\text{path distance}} - P_{\text{dead-end}} \tag{1}$$

Reachability is used to check whether most walkable cells are connected. Wall ratio controls the proportion of walls and compares it with the target value of 0.28. Junction complexity measures the number of branches and intersections, which can provide more possible route choices during the chase. Path distance is used to evaluate whether the average BFS path length in the map is suitable for the survival chase setting. The dead-end penalty is added to reduce the number of cells with very few exits, because too many dead ends may make the map unfair or less playable.

During each generation, the generator calculates the fitness score of every map in the current population. Then, maps with better scores are selected as parents through tournament selection. The crossover operation combines partial structures from two parent maps to generate offspring maps. The mutation operation randomly changes a small number of cells with a low probability, such as changing walkable ground into walls or changing walls into walkable ground. After several generations of selection, crossover, and mutation, the system gradually evolves maps that are more suitable for the game environment.

At the end, the map with the highest fitness score is saved as a text file. The map file uses a space-separated `0` and `1` format, where each line corresponds to one row of the map. This format is simple, readable, and easy for both Python and C++ game modules to load. Apart from the map grid itself, the system also validates the spawn points of the human and monsters. The spawn points must be located on walkable cells, and the human and monster spawn points cannot overlap. The system also checks the BFS path distance between them. In this implementation, the valid starting distance is set between 10 and 25 BFS steps, so that the monster is not placed too close or too far away from the human at the beginning of the game.

2.2 Human AI Algorithm (BFS)

When the user plays the role of monster, a Breadth-First Search(BFS)-based system is used to control the human character. Generally, the objective of the human character is to keep as far away from the monster as possible. Therefore, every time when the Human AI needs to decide its move direction, we run BFS starting from the monster's current position. This creates a distance map over the whole maze, where each reachable cell stores how many steps the monster would need to reach it. Since the maze is an unweighted grid, BFS gives the shortest path distance correctly.

After building this distance map, the Human AI checks the four possible next positions of the human: up, down, left, and right. It chooses the move with the largest BFS distance from the monster, meaning the human tries to move to the neighboring cell that is farthest away in terms of actual maze path distance.

2.3 Monster Algorithm

2.3.1 A* Search Algorithm

A* is a heuristic graph search algorithm. It finds the shortest path between two nodes by adding the cost of the lined path to the residual cost estimate of reaching the target node. The cost of each node is calculated by the following function:

$$f(n) = g(n) + h(n)$$

In the process of the monster moving to the player in each round, the map grid is regarded as an untitled map, thus calculating the path cost. Due to randomly generated roadblocks, each walkable grid cell has up to four neighbors, which can be reached by moving up, down, left or right. When calculating heuristic functions, we use the Manhattan distance between the current cell and the player's location. In addition, because the map has a ring structure, we will apply modular operations to the row index and column index every time we expand the neighbor. This allows movement to cross the grid boundary. After the algorithm is executed, the core function astar(grid, start, goal) will return the full path from the current position of the monster to the player. With each move of the player, the Astar algorithm will calculate the corresponding optimal path. If there is no path (for example, the player is completely surrounded by a wall), the monster will stay in place.

2.3.2 Greedy Search for Monster Movement

The Greedy Search is our basic pursuit strategy, which applied in the "simple" level. At each beginning of each round, our system will run a BFS computation at the current cell, in order to build a distance filed: the array stores the real distance between player and monster (with grid steps, including the circumstance of walls and wrap-around) to human. After that, each monster check all the cell it can move to, and choose the shortest one, and repeat it again. The system avoid two monster stayed in the same cell. If the monster is stuck, and have no possible way to player, we will use Manhattan distance, so the monster still makes forward progress.

The Greedy Search is very speedy and simple, each round and only need to compute BFS once and a constant number of comparison. Its shortcoming is that the monster is totally passive during the game, can not predict where the player would go. Facing

the clever player, it would be dumb and be stuck in the trap, so it will be more long for it to catch the player. Hence, the Greedy Search is a well-known basic strategy, the foundation of the higher or cleverer algorithm.

2.3.3 Simulated Annealing Algorithm

At the third difficulty level, Simulated Annealing (SA) is used to plan the monster's movement. Unlike A* or BFS, SA does not require every intermediate candidate to be a wall-free shortest path. Instead, it searches over complete, continuous routes from the monster to the human and assigns large costs to routes that cross walls. This representation makes mutations inexpensive while allowing the algorithm to explore routes that differ substantially from the current one. Since the monster executes only the first two steps of a route before replanning, the objective gives particular importance to the beginning of the path and a wall-aware BFS is retained as a final safety mechanism.

The maze is treated as a torus. For two cells $a = (a_x, a_y)$ and $b = (b_x, b_y)$, their heuristic distance is

$$Dist(a, b) = \min(|a_x - b_x|, 30 - |a_x - b_x|) + \min(|a_y - b_y|, 30 - |a_y - b_y|).$$

The initial state is a random toroidal Manhattan shortest path from the monster to the human. The required horizontal and vertical moves are calculated first and then shuffled. When two wrap-around directions have the same length, one is selected randomly. Consequently, the initial path is continuous and always ends at the human, but it deliberately ignores walls. Wall avoidance is then introduced through mutation and the energy function.

To generate a neighbouring state, the algorithm selects two points on the current path and rebuilds the segment between them. With probability 0.70, the left endpoint is fixed at the monster's current position so that the mutation can change the two steps that will actually be executed. Half of these early mutations rebuild the path all the way to the human; the other half use a right endpoint no more than 30 path positions away. For a non-early mutation, the left endpoint is sampled uniformly from the path and the right endpoint is selected within the same 30-position span.

The replacement segment is constructed by a wall-aware greedy search for at most 80 steps. At each step, it chooses a walkable neighbour with minimum toroidal Manhattan distance to the segment endpoint. The algorithm prefers unvisited cells and moves that do not immediately backtrack. Meanwhile, candidates with the same expected distance are shuffled to preserve variation. If the greedy search does not reach the right endpoint within a certain number of steps, a random toroidal shortest path is appended to close the segment. This closing part may cross walls, but the resulting route must remain continuous, start at the monster, end at the human, and contain no more than 140 cells. A candidate that fails these structural checks is rejected immediately.

For a path $P = (p_0, p_1, \ldots, p_{n-1})$, let h be the human position and $p_{\text{move}} = p_{\min(2,n-1)}$ be the position reached after the move executed in the current turn. The energy minimized by SA is

$$E(P) = |P| + 3Dist(p_{\text{move}}, h) + Wall(P) + R(P).$$

The path-size term favours short routes, and the larger weight on $Dist(p_{\text{move}}, h)$ encourages the monster to move towards the human to maintain pursuit pressure.

The wall cost distinguishes cells near the executable part of the route from those farther in the future:

$$Wall(P) = \sum_{i=1}^{n-1} \begin{cases} 10000, & p_i \text{ is a wall and } i \leq 6, \\ 100, & p_i \text{ is a wall and } i > 6, \\ 0, & p_i \text{ is walkable.} \end{cases}$$

The very large early penalty strongly penalizes an illegal current move. The smaller later penalty still guides the search towards a fully walkable path, while also permits SA to pass through temporarily poor intermediate states. Because a new route is planned every turn, the first six steps are attached more importance than a distant suffix that may not be executed instantly.

The final term reduces visible repeatation between turns:

$$R(P) = 10I(p_{\text{move}} = p_{\text{prev,start}}),$$

where $p_{\text{prev,start}}$ denotes the monster position at the beginning of the previous round. The game backend records the previous movement and passes it to the C++ planner on the next turn. Thus, returning immediately to the previous position is discouraged without preventing the monster from continuing a useful direction of pursuit.

A candidate P' with improvement is always accepted. A worse candidate might also be accepted with probability

$$\Pr(P \to P') = \exp\left(\frac{E(P) - E(P')}{T}\right).$$

This permits broad exploration at high temperature and increasingly selective search as the temperature falls. The implementation also tracks the best accepted route whose first two steps are both walkable. That route is prefered over a lower-scoring route that the monster cannot execute safely.

Table 1: Default simulated annealing parameters

Parameter	Value
Candidate trials per temperature	20
Initial temperature	80
Minimum temperature	0.5
Cooling rate	0.94
Immediate-move distance weight	3
Normal wall penalty	100
Wall penalty in the first six steps	10000
Greedy segment-rebuild budget	80 steps
Maximum local segment span	30 path positions
Probability of an early mutation	0.70
Maximum candidate path size	140 cells

With these defaults, the temperatures are $80, 80(0.94), 80(0.94)^2, \ldots$ while $T > 0.5$. This gives 83 temperature levels and $83 \times 20 = 1660$ candidate mutations per call.

After annealing, the monster chooses the best accepted route with legal first steps. If no such route is found, a wall-aware toroidal BFS constructs a safe fallback path. The selected route is checked again before output, ensuring that the monster never crosses the wall even though SA is allowed to evaluate wall-crossing candidates during the search.

The current configuration was chosen based on an ablation experiment. We tested the algorithm on five generated maps with five fixed random seeds per map, it caught the human in 24 of 25 games. Its mean planning time was 6.72 ms, the 95th percentile was 10.16 ms, and the maximum was 12.35 ms. No BFS fallback was required and the selected routes contained no wall visits in these runs. These results show that the method is fast enough for interactive play. Also, the algorithm can effectively retain safe movement while producing varied routes.

2.3.4 Mininax for Predictive Monster AI

Minimax models the chasing process of monster and player as a zero-sum game. The monster as the Min, wanting to shorten the distance between one another, while the player is the Mac, wanting to make the distance larger. The system builds a depth-limit game tree which represent every step or action the agent might move. Following the round-round structure, at each round, the monster will search and take two step forward, then play will take one-step action. The round will be repeated with a constant number rounds, the default depth is two, so that it will be within the computation budget.

Leaf nodes are calculated by an evaluation functions, which is based on the distance of player to each monsters.

$$\text{Eval} = 4 \cdot \min_i d(\text{Human}, \text{Monster}_i) + \sum_i d(\text{Human}, \text{Monster}_i)$$

We give the first term as a approximately weight 4, which is the biggest weight as well, since the smallest distance represent the biggest risk. The summation keeps the other monsters moving as well. Small figure favor the monsters, while the big one favor the player. If the monster get the player's cell, the node will return a big negative score, and if the use step is the score is lower, so that the monster will try to use the least step to catch.

Searching the whole tree is expensive, so we use standard optimizations. First, we use the alpha-beta pruning, to eliminate the branch which is impossible to effect the final decision, so that we can cut down the quantity of the nodes to be searched. Second, we apply the transposition table to cash the result of each evaluated state. With player position, tuple of monsters position, and ply index as keys and all together with a bound flag. Since all in a wrap-around grid, so we can avoid a lot of repeated work and calculation. Third, we use the move ordering to check the most hopeful path first: the path of monster are ordered by increasing distance to player, the path of monster are ordered by decreasing distance to player, and the best path recorded by the transposition table. Good sequence will make the alpha-beta prune much more aggressively and get a better result. After the searching, the chosen sequence of move for current move will be rebuilt from the transposition-table entries and returned through the same interface.

Compared to Greedy Search, Minimax and do predict to the player, more predictable. Since it will modify simulate the best choice made by player, so it will do cut the escape route of the player, but not just follow the player passively. But the cost is high computation, and increasing with branching factor and look-ahead depth. The optimization

above is the key factor that make the algorithm into usable. Hence, Minimax will be used in the more difficult level.

2.4 Machine Learning Dual-Monster Mode

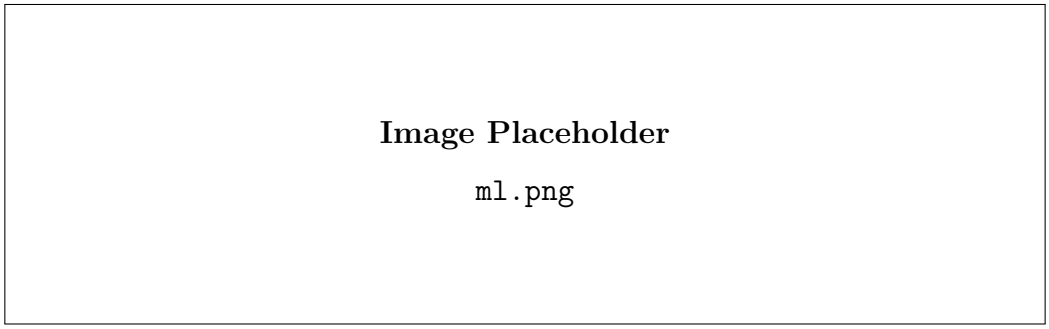


Figure 2: Our Work

Listing 1: Serialized JSON Structure of the Gameplay Instance

```
{
  "features": {
    "rel_row": -3,
    "rel_col": 5,
    "wall_up": 0,
    "wall_down": 1,
    "wall_left": 0,
    "wall_right": 0,
    "prev1": 2,
    "prev2": 1
  },
  "label": "left"
}
```

To achieve efficient collaborative pursuit in the dual-monster mode, this project introduces a machine learning approach to predict the human player's next movement direction. Given that the game map represents a non-linear discrete space, we selected a Random Forest classifier to construct the predictive model . The training data was collected entirely through real gameplay testing conducted by our group members, resulting in a total of 4,206 data entries saved in JSONL format. Each line represents a player decision sample containing the current state features and the ground-truth movement label. Based on this dataset, to minimize the parameter count while preserving high-quality characteristics, the model extracts and learns three core categories of features:

1. **Spatial Relative Position:** The two-dimensional coordinates of the human relative to the monster, using the monster as the reference frame.

2. **Environmental Obstacles:** Binarized features indicating whether the human is blocked by walls in the four adjacent directions.

3. **Historical Behavior:** The human player's movement directions during the previous two actions.

The output target of the model is the human's movement direction in the next step.

During training, all record data is first loaded from the file `replay_log.json`, and then each record is converted into a corresponding feature matrix and label vector. Subsequently, the dataset is divided into an 80% training set and a 20% test set, and fed into a **random forest classifier** for training. Table 2 details the hyperparameter configuration of our **random forest model**. Experimental results show that the model exhibits excellent generalization ability on the test set, achieving an accuracy of 77.08% in predicting the next direction.

Table 2: Random Forest Hyperparameters

Hyperparameter	Value	Description
`n_estimators`	300	Number of decision trees
`max_depth`	12	Maximum depth of each tree
`min_samples_leaf`	3	Minimum samples at a leaf node
`class_weight`	balanced	Handles class imbalance
`random_state`	42	Ensures reproducibility

In the dual-monster mode, one monster uses the A* algorithm to directly track the player's current position, while the other monster uses the player's predicted position as the target point.

$$
\begin{aligned}
p_{t+1} &= p_t + \text{move}(\hat{d}_{t+1}) \\
p_{t+2} &= p_{t+1} + \text{move}(\hat{d}_{t+2}) \\
p_{t+3} &= p_{t+2} + \text{move}(\hat{d}_{t+3})
\end{aligned}
$$

where p_t denotes the player's current coordinate, and $\hat{d}_{t+1}$, $\hat{d}_{t+2}$, and $\hat{d}_{t+3}$ represent the predicted movement directions for the next three consecutive steps, respectively. The final obtained coordinate p_{t+3} is designated as the target point for the second monster (the interceptor). Through practical gameplay testing, the average spatial prediction error for the future third-step position is only around 2 grid cells. This anticipation mechanism allows the interceptor to seal off the human player's escape routes in advance, creating a highly effective collaborative pincer attack alongside the primary monster that directly pursues using the A* algorithm.

3 Game System Design and Gameplay Integration

3.1 Overall Game Flow

3.1.1 Game Execution Process

The complete game flow of *Maze Escape* can be divided into three stages: initialization, turn-based gameplay, and end-game evaluation with reset. Together, these stages form a continuous cycle that starts when the game launches, ends when a win or loss condition is reached, and then returns to the beginning for a new game session.

1. Initialization Stage After selecting Start Game, the player first chooses a character on the Character page. If the player selects Human, a difficulty level must be chosen before entering the game. If Monster is selected, the game starts immediately.

Before the game begins, the system uses a genetic algorithm to generate the maze, determine the initial item spawn locations, and set the starting positions of both the Human and the Monster. A minimum safe distance is maintained between them to prevent the Human from being caught immediately at the start of the game.

2. Turn-Based Gameplay The game follows a fixed turn order. In each round, the player-controlled character always moves first, followed by the AI-controlled character. The Human can move one tile per turn, while the Monster can move two tiles per turn.

After every movement, the character's position is updated and the step count is synchronized with the HUD. The system then checks whether the Human and Monster occupy the same position. This check serves as the connection between the turn-based gameplay loop and the end-game condition.

When playing in Human mode, the system also checks whether the Human has stepped on an item. If so, the corresponding item effect is triggered and integrated into the gameplay.

3. End-Game Evaluation and Reset The game ends once the Human and Monster occupy the same position on the map. The system then switches to the End page.

On the End page, players can choose "Replay", which generates a new map while keeping the same game mode and difficulty settings, or "Return to Main Menu" to go back to the start screen.

After the reset is completed, the game returns to Stage (1), creating a complete gameplay cycle.

3.1.2 Interface Navigation and Global Button Rules

From the frontend perspective, the game flow is reflected through page transitions between different interfaces. The navigation sequence is:

Index → Character Select → (Human passes through Difficulty Selection, Monster skips this step) → Game → End

The following flowchart illustrates the complete navigation process:

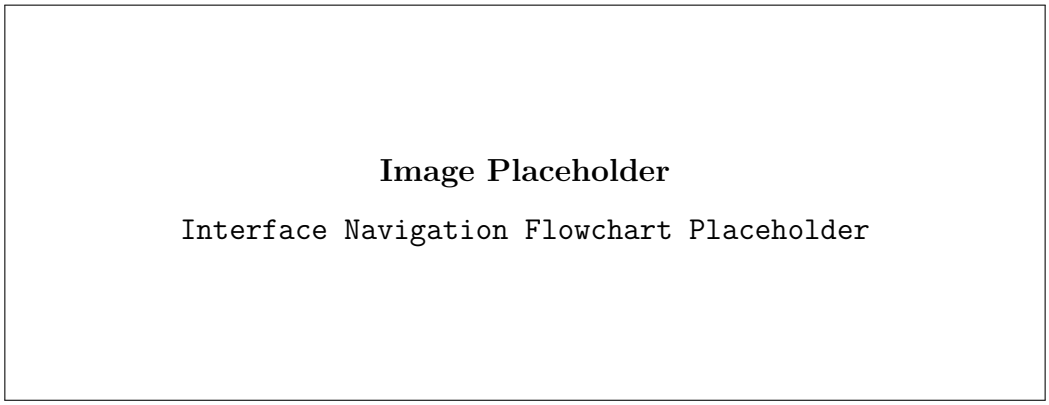


Figure 3: Interface navigation flowchart

Except for the Index page, all other interfaces support two global shortcut keys:

- Press **M** to open the Manual overlay, which contains game instructions and the item guide. The overlay can be closed using the button in the top-right corner without leaving the current page.
- Press **ESC** to return directly to the Index page.

These shortcut key instructions are also included in the Manual for player reference.

3.2 Game State Representation

The game state is centrally managed through a `GameState` data class. Each game session is represented by an independent instance and stored in the global `games` dictionary using `game_id` as the key. This design allows multiple game sessions to run simultaneously.

1. Grid and Coordinate System The map is represented as a two-dimensional matrix, where each cell is classified as either a walkable path or an unwalkable wall. In addition, the map uses a wrap-around boundary design. When a player or monster moves beyond one edge of the map, it reappears on the opposite side instead of being blocked by the boundary.

The positions of the player, monster, and items are not stored directly within the grid. Instead, they are maintained separately as independent game data.

2. Entity States The player state mainly records the current position of the player and any temporary effects gained from collected items. The monster state records its current position and whether it is currently affected by any item-related effects.

3. Mode and Algorithm Binding The algorithms used for different game modes and difficulty levels are managed through a separate scheduling mechanism. The selected algorithm is displayed on the HUD when the game starts. This design keeps the game state structure independent of any specific algorithm implementation.

4. Item and Effect States Before being collected, items exist as entities placed on the map. Once collected, they are converted into temporary status effects applied to either the player or the monster. All items take effect immediately after pickup, and their effects gradually decrease as turns progress.

3.3 Turn-Based Movement Logic

Based on the fixed turn order described in Section 3.1, this section explains the detailed movement rules used during gameplay.

By default, the player can move one tile per turn, while the monster can move two tiles per turn. Before each movement, the system checks whether the target tile is walkable. If the tile is blocked by a wall, the movement is rejected and the game state remains unchanged.

Collision and capture checks are performed after every single-tile movement rather than only once at the end of a turn. This allows the game to detect captures immediately when they occur.

Since the number of player movements can be affected by item effects, the system does not assume a fixed number of moves per turn. Instead, it continuously checks whether the player still has remaining movement points in the current turn. The monster only begins its turn after the player has used all available moves or when a special effect ends the player's movement phase early.

The monster's two-tile movement is executed step by step rather than as a single jump. After each tile movement, the system checks again whether the monster and player occupy the same position. If the player is captured after the first step, the second step is not executed. This step-by-step design also allows the frontend to render intermediate monster positions, resulting in smoother movement animations.

3.4 Interactive Item System

The item system within this project was designed as a significant component of the game logic, rather than merely a decoration on the map. Since the monsters can chase the player using algorithms such as A* Search, Greedy Search, and machine-learning-based prediction models, the game would become too difficult if the player could only move one step per turn. Therefore, the main purpose of the item system is to provide short-term escaping, defensive, and repositioning abilities, which improves the playability and balance of the game.

This system includes four items: **Swift Boots, Frost Trap, Teleport Stone, and Cloak**. These four items affect the player's movement ability, the monsters' movement status, the player's location, and the monster-chasing logic respectively. All item effects are integrated with the turn-based movement system. In most cases, the duration of an item effect is updated based on a complete player turn, rather than based on a single movement cell.

3.4.1 Swift Boots

Swift Boots is a movement-enhancing item. After the player picks up this item, the player can perform at most two separate one-cell movements in each of the following five complete player turns. These two movement steps can be in different directions. For example, the player can first move upward and then move to the right. Therefore, this item does not simply allow the player to move two cells in a straight line. Instead, it changes the player's turn structure and allows the player to perform multiple movements within one turn, similar to the movement pattern of the monsters.

This design is more flexible than directly moving two cells in one direction, because the player can turn corners to avoid walls or escape from the monster's chasing path. Each single-cell movement is still checked independently for wall collision, item pickup, and monster collision. As a result, the player cannot pass through walls or skip important events on the map.

The duration of Swift Boots decreases by one only after the player finishes a complete turn, instead of decreasing after each single movement. If the player picks up another Swift Boots while the effect is still active, the duration will be refreshed to five turns instead of being stacked repeatedly. This mechanism ensures that the item is useful enough for escaping, while preventing the player from staying in an overly powerful state for too long.

3.4.2 Frost Trap

Frost Trap is used when the player needs a short break from the monsters. In our game, the monsters can move very aggressively, especially when two monsters are enabled. After the player picks up Frost Trap, every monster is given a frozen state for five player turns. While this value is still greater than zero, the monster stays where it is and its movement step is skipped.

We made the item affect all monsters instead of only one monster. This is because freezing only one monster does not really solve the problem in dual-monster mode. The other monster could still keep moving toward the player, so the player may still be caught very quickly. By stopping all monsters at the same time, the player gets a few turns to move out of a bad position, change route, or try to reach another useful item.

The freeze effect is not allowed to stack forever. If the player gets another Frost Trap while the monsters are still frozen, the game does not keep adding five more turns each time. The frozen counter is refreshed or kept at the larger value. This makes Frost Trap useful in dangerous moments, but it does not let the player freeze the monsters for too long.

3.4.3 Teleport Stone

Teleport Stone is used when the player is trapped or too close to the monsters. After the player picks it up, the player is not moved to a completely random place. We avoided pure random teleportation because it could send the player to a cell that is still near a monster, which would make the item almost useless.

In the actual implementation, the game checks the map and builds a list of possible teleport positions. The target cell must be walkable, and it cannot be the player's current cell, a monster cell, an active item cell, or an active trap cell. After these invalid cells are removed, the game calculates how far each remaining cell is from the nearest monster.

We then use this distance to decide which cells are safer. Cells farther away from the nearest monster are treated as better teleport choices. Instead of always choosing the single farthest cell, the game randomly chooses one position from the top 10% safest valid cells. This makes the item safer than random teleportation, but the result still has some uncertainty.

This rule is useful in the two-monster setting. If the player only moves away from one monster, the other monster may still be close. Since the teleport position is judged by the distance to the nearest monster, the selected cell is usually not too close to either monster. This gives the player a better chance to escape when the monsters are coming from different sides.

3.4.4 Cloak

Cloak is a stealth-type item. When the player picks up this item, the player becomes invisible for ten complete player turns. During invisibility, monsters cannot detect or capture the player. Instead of using A* Search, Greedy Search, or machine-learning-based prediction to chase the player, the monsters move randomly among valid neighbouring cells.

This design changes the behaviour of the monsters during the invisible state. The monsters are still active in the maze, but they no longer have a clear target. Therefore, the game does not become completely static when the player is invisible. At the same time,

even if a monster randomly moves onto the player's position, the game-over condition will not be triggered while the invisibility effect is active.

The duration of Cloak is also updated based on complete player turns. This is important because Swift Boots may allow the player to perform two movement steps within one turn. In this case, the Cloak duration should still decrease by only one after the whole player turn is completed, rather than decreasing twice. This keeps the time-based logic of all item effects consistent.

After the invisibility duration ends, the monsters return to their normal chasing behaviour and collision detection becomes active again. Since Cloak can completely prevent the player from being captured in a short time, it is designed as a rare and powerful item.

3.4.5 Item-State Update and Game Balance

In our game, item effects are counted by player turns rather than every single movement. This is important because Swift Boots can let the player move twice in one turn. If the effect duration were reduced after each movement step, Swift Boots and Cloak would lose time too quickly and the item logic would become inconsistent. Therefore, the remaining turns of Swift Boots, Frost Trap, and Cloak are updated only after the player finishes the whole turn.

The item system also makes the game fairer for the human player. The monsters can chase the player with search algorithms and prediction models, so the player needs some short-term abilities to survive. Swift Boots gives extra movement, Frost Trap stops the monsters for a short time, Teleport Stone moves the player to a safer position, and Cloak prevents the monsters from catching the player temporarily. These items do not remove the challenge of the game, but they give the player more choices when the monster AI becomes too strong.

Overall, the interactive item system increases the complexity and playability of the game. Swift Boots improves the player's mobility, Frost Trap provides temporary control over the monsters, Teleport Stone allows emergency repositioning, and Cloak temporarily prevents the monsters from targeting the player. These items work together with the turn-based game logic and make the maze escape process more strategic and dynamic.

Table 3: Summary of Interactive Item Effects

Item	**Type**	**Effect**
Swift Boots	Mobility	Allows the player to perform two separate one-cell movements per turn for five complete player turns. The two movement steps can be in different directions.
Frost Trap	Control	Immediately freezes all monsters for five complete player turns. Frozen monsters cannot move or continue chasing the player.
Teleport Stone	Escape	Teleports the player to a randomly selected cell from the top 10% safest valid cells, based on the distance to the nearest monster.
Cloak	Stealth	Makes the player invisible for ten complete player turns. During invisibility, monsters move randomly and cannot capture the player.

3.5 User Interface and Backend Integration

The entire game interface adopts a unified pixel-art forest theme inspired by the visual style of *Stardew Valley*. Beyond maintaining a consistent visual appearance, the frontend and backend are closely integrated through continuous data synchronization. Every player action sends a request to the Flask backend, which updates the game state and returns serialized data. The frontend then uses this data to refresh and re-render the interface.

The following sections introduce the main interface components and explain how synchronization between the frontend and backend is achieved.

3.5.1 Pre-Game Pages

The Index page serves as the main entry point of the game. It features a wooden signboard-style title design and provides three main options: Start Game, Manual, and Exit.

The Manual page provides detailed information about the game, including the differences between the two game modes, control instructions, and the functions of the four available items. To make the information easily accessible during gameplay, players can press the **M** key on any page to open the Manual as an overlay. This allows them to quickly review the controls and item effects without leaving the current game session.

The Character Selection interface uses a left-right card layout to present the two playable roles: Monster and Human, each illustrated with pixel-style artwork. When the mouse hovers over a character card, the card becomes highlighted, providing visual feedback to indicate the currently selected option. After the player clicks a role, a loading animation is played immediately to enhance immersion.

The Difficulty Selection interface only appears when the player chooses the Human mode, while Monster mode skips this step and enters the game directly. This page follows the same hover-to-highlight interaction pattern as the character selection screen, but uses

different highlight colors to distinguish difficulty levels more clearly. After selecting a difficulty, a confirmation popup appears. If the player chooses "No", the popup closes and the player remains on the current difficulty selection screen, improving error tolerance and overall user experience.

Although the pre-game interface appears to be mainly based on static visual interactions, it actually relies heavily on backend communication to initialize each game session.

After the player completes the configuration in the Character Selection and Difficulty Selection screens, the frontend sends the selected parameters—such as game mode (`mode`), opponent AI type (`opponent_ai`), and number of monsters—via a POST request to the backend API.

Upon receiving the request, the backend generates a random maze using a dedicated function, initializes the monster pathfinding logic according to the selected difficulty and algorithm type, and sets up the initial positions of both the player and monsters, as well as item distribution across the map.

Once initialization is complete, the backend uses the `GameState.serialize()` method to convert the entire game state into JSON format and returns it to the frontend. The frontend then renders the initial gameplay screen based on this data.

In this way, the pre-game interaction pages act as a bridge between user configuration and backend session initialization, ensuring that different modes and difficulty levels correctly produce distinct and consistent game environments.

3.5.2 Gameplay Interface

The Game page continues the overall pixel-art visual style. The map area on the left is rendered using a Canvas, while static wall elements are pre-rendered using an offscreen canvas to reduce repeated drawing costs in each frame.

Player movement animations are implemented using linear interpolation (lerp) for smooth transitions. During animation playback, an `inputLocked` state is used to disable new inputs, preventing incoming commands from interrupting ongoing animations.

On the right side, the HUD panel consists of two modules: Game Stats and Item Panel.

The Game Stats module displays real-time information such as the current difficulty level, step counts for both the Human and Monster, and item usage counts. The Item Panel shows four core items as icon cards: Speed Boots, Frost Trap, Teleport Stone, and Invisibility Cloak. Beneath each item card, corresponding status information is displayed, including remaining duration of speed effects, whether the player is currently invisible, remaining frozen turns for the monster, and item respawn cooldown timers. All these values are continuously synchronized from backend state updates, allowing players to understand the game situation clearly without interrupting gameplay.

This interface represents the most interaction-intensive part of the system, where the frontend and backend continuously synchronize in a turn-based manner. In Human mode, each movement command sent by the player includes a direction parameter, which is processed by the backend through the `apply_player_move()` function. The backend first validates whether the target grid is walkable. If the move is valid, it further checks whether an item exists at that location and triggers the corresponding effect.

Due to the presence of the Speed Boots item, the number of movements per turn is not fixed at one step. Instead, the system dynamically determines whether the player can continue moving using state variables such as `remaining_player_steps_this_turn` and

`waiting_for_second_step`. These variables control whether the turn should continue or wait for additional input before ending.

Once the player's turn is completed, the backend computes the monster's next position using a pathfinding algorithm based on the selected difficulty. It also updates all time-based effects, such as item duration and freeze counters. The complete updated state is then returned via `serialize()`, allowing the frontend to refresh both the map and HUD simultaneously.

In comparison, the Monster mode logic is simpler. Since this mode does not involve the item system, each player movement only requires path validation and coordinate updates on the backend. After every two moves, a BFS-based algorithm is triggered to update the human's automated movement. This loop continues until both entities meet, resulting in lower interaction frequency and reduced system complexity compared to Human mode.

Additionally, a collision warning popup is implemented as part of frontend-backend coordination for handling invalid actions. When a player attempts to move into a wall, the backend checks the target grid inside `move_one()` and `move_human_with_items()`. If the move is invalid, the backend returns an error response. The frontend then displays a popup based on this response, turning backend validation results into immediate visual feedback and preventing user confusion caused by unresponsive inputs.

4 Validation and Evaluation

4.1 Map Generation and Spawn Test

For the map generation part, we checked whether the output map could be used in the game. A map that appears correct in the text file may still be unsuitable for gameplay. It also needs to support movement, chasing, and valid spawn positions. Therefore, we checked the map format, connectivity, wall density, and spawn distance before using the map in the main game system.

The first check is the map format. The game uses a fixed 30×30 grid, so the generated file must contain 30 rows and 30 columns. Each cell must also be either `0` or `1`. In our map file, `0` represents a walkable tile, and `1` represents a wall. This check is important because the backend movement logic and the frontend canvas rendering both depend on this fixed format.

The second check is map connectivity. A map may have the correct size, but the walkable area may still be separated into different regions. If this happens, the human and monsters may not be able to reach each other. To avoid this problem, we used BFS to check whether the walkable cells were sufficiently connected. The game uses wrap-around movement, so the BFS check follows the same neighbour rule as the actual movement logic.

We also checked the wall density. In this project, the target wall ratio is about 0.28. This value gives the map enough obstacles without making the maze too blocked. If there are too few walls, the monster can chase the human too directly. If there are too many walls, the movement space becomes too limited. For this reason, the wall ratio is included in the fitness function to help control the difficulty of the generated map.

After checking the map structure, we checked the spawn positions. The human and monsters must start on walkable cells. Their starting positions must not overlap. We used BFS path distance to measure the distance between the human and the monster, rather

than using Manhattan distance. This is because the actual path is affected by walls and wrap-around movement. In the final version, the starting distance is controlled between 10 and 25 BFS steps. This prevents the monster from catching the human immediately, but still keeps the chase active from the beginning.

Finally, the generated map is saved as `map/generated_map.txt` and loaded into the main game program. After loading the file, we tested whether the human could move normally, whether the monsters could chase through the maze, and whether collision detection worked correctly. If these functions operate correctly on the generated map, the map generation module can be considered successfully integrated with the game system. These checks make the output map more than a random grid. It becomes a playable maze that supports spawning, movement, chasing, and game-state updates.

4.2 Algorithm Evaluation

To evaluate the performance of different algorithms from multiple perspectives, we conducted a series of experiments on four algorithms using a maze escape game as the platform, comparing their performance with the algorithm type as the sole independent variable. For each algorithm, we ran 30 games under the same game configuration, including the same map generation process, grid size, movement rules, spawn point distance logic, and maximum move limit. This ensures that any performance differences are attributable to algorithm-driven decision-making. Below, we will detail the evaluation experiments and their results.

This evaluation considered three metrics: capture rate, capture steps (the number of steps required to capture the player), and average decision time. First, we used the capture rate to measure the algorithm's stability: for capture steps, a more powerful algorithm-driven monster should be able to successfully capture the player within a fewer step limit. Second, fewer capture steps are not necessarily better; a balance must be struck between capture efficiency and pursuit efficiency. Therefore, we finally calculated the decision time to compare the computational cost required for each decision by different algorithms.

- **Catch Rate:** According to the experimental results (as shown in Figure 5), A*, Greedy, and Minimax all achieved a 100% catch rate, demonstrating the high reliability of these three algorithms in a maze environment like maze-escape. However, the catch rate of Simulated Annealing (SA) was only 83.3%. Tracing this back to the underlying principles of the algorithm, we found that because SA is based on probabilistic optimization, it accepts certain sub-optimal moves during the decision-making process to explore other potential paths. While this mechanism offers advantages in certain optimization problems, it is detrimental in real-time pursuit scenarios. Accepting poor solutions introduces randomness into the monster's behavior, preventing it from catching the player within short-duration chases. This also indicates that deterministic search-based algorithms are better suited for real-time maze escape games than simulated annealing.

- **Capture Steps:** Among the four algorithms, Minimax achieved the best results, requiring an average of only 20.5 steps to capture the player (the specific distribution is shown in Figure 4). Greedy followed closely behind with an average of 20.8 steps, while A* required an average of 21.3 steps. The differences among these

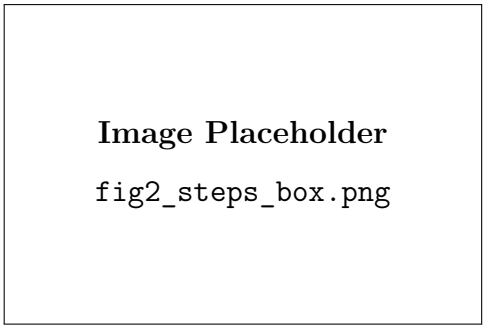


Figure 4: Steps Distribution

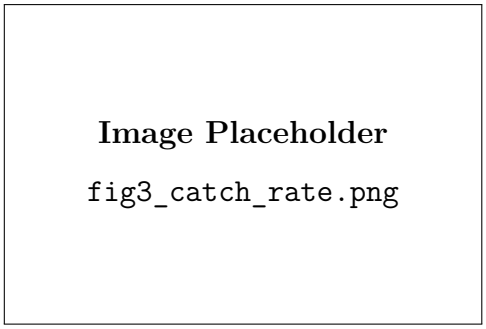


Figure 5: Catch Rate

Figure 6: Performance Overview

three algorithms are minimal, and they all share a median of 20 steps. This demonstrates that A*, Greedy, and Minimax are all capable of generating highly efficient pursuit behaviors in the maze. Based on the principles of the Minimax algorithm, we analyzed why it slightly outperforms the others: Minimax not only considers the player's current position but also uses lookahead search to preemptively restrict the player's future escape options. SA showed the weakest step performance. In successfully captured games, its average capture steps reached 26.8, which is significantly higher than the other three algorithms for the reasons mentioned above.

- **Decision Time:**

Table 4: Performance Comparison of Different Algorithms

Algo	Caught	AvgTime (ms)	MedTime (ms)
A*	30/30	0.0511	0.0342
Greedy	30/30	0.3449	0.3184
Minimax	30/30	0.6211	0.6299
SA	25/30	9.9962	9.8462

Considering that our game is a real-time interactive system, the AI must not only possess strong pursuit capabilities but also guarantee sufficient response speed. A* achieved the fastest decision speed, requiring an average of only 0.0511 milliseconds per decision. From an operational efficiency standpoint, it is the most outstanding among the four algorithms. Greedy's average decision time was 0.3449 milliseconds; although slightly slower than A*, it remains highly efficient. Minimax recorded an average decision time of 0.6211 milliseconds. Compared to A* and Greedy, its

execution speed is slower, but the reason is evident: Minimax needs to search future possible game states before making a decision. SA performed poorly, with an average decision time as high as 9.9962 milliseconds, far exceeding the other three algorithms.

- **Comprehensive Performance Analysis:** We aim to find the optimal trade-off between capture steps (efficiency) and decision time (speed), where the ideal algorithm should sit as close as possible to the bottom-left region of the chart **??**. The chart shows that the A* algorithm delivers the best comprehensive performance, maintaining a low capture step count while delivering ultra-fast response times. Consequently, we deployed the A* algorithm in the game's hardest levels, and the subsequent machine learning dual-monster mode was also developed based on A*. Minimax and Greedy represent two alternative solutions that achieve a good balance between capture steps and running speed. In contrast, Simulated Annealing performed the worst, lagging significantly behind the others in both aspects.

In conclusion, due to its exceptional execution speed, A* is the most outstanding all-round algorithm; however, when minimizing capture steps is the primary goal, Minimax performs the best. Conversely, the SA algorithm is entirely unsuitable for application in real-time pursuit games.

4.3 Machine Learning Evaluation

To evaluate whether the random forest model truly learned the characteristics of human movement behavior, we analyzed the model from two dimensions: feature importance and ablation study. Through these two analyses, we not only measured prediction accuracy but also understood which type of information contributed the most to predicting player behavior. Regarding the model's prediction performance, the confusion matrix heatmap provides a comprehensive classification view. The model performs best when predicting "left" with a recall of 0.81, followed by "down" at 0.79, "right" at 0.75, and "up" at 0.71. This demonstrates that the model correctly predicts the vast majority of player movements across all four directions without any extreme bias.

To evaluate the performance of different algorithms from multiple perspectives, we conducted a series of experiments on four algorithms using a maze escape game as the platform, comparing their performance with the algorithm type as the sole independent variable. For each algorithm, we ran 30 games under the same game configuration, including the same map generation process, grid size, movement rules, spawn point distance logic, and maximum move limit. This ensures that any performance differences are attributable to algorithm-driven decision-making. Below, we will detail the evaluation experiments and their results.

This evaluation considered three metrics: capture rate, capture steps (the number of steps required to capture the player), and average decision time. First, we used the capture rate to measure the algorithm's stability: for capture steps, a more powerful algorithm-driven monster should be able to successfully capture the player within a fewer step limit. Second, fewer capture steps are not necessarily better; a balance must be struck between capture efficiency and pursuit efficiency. Therefore, we finally calculated the decision time to compare the computational cost required for each decision by different algorithms.

First, according to the feature importance analysis of the random forest model, it is evident that the model does not rely solely on a single variable; spatial position, obstacle information, and historical movement information are equally essential. According to the feature importance chart (Figure 7b), historical movement stands out as the most critical feature. Among them, `prev1` (the previous step's movement) yields an importance score of 0.2842, followed by `prev2` (the movement two steps prior) at 0.1463. This indicates that the player's most recent movements strongly influence the prediction of the next step, reflecting that in actual gameplay scenarios, players often continuously move in the same direction when escaping. In addition, spatial features are also key: `rel_col` (relative column distance) holds an importance of 0.1443, and `rel_row` (relative row distance) stands at 0.1211. These two features describe the player's position relative to the monster, enabling the model to learn how players react when the monster approaches from different directions. Lastly, wall features exhibit a lower importance, hovering around 0.07 to 0.08, which suggests that obstacle features primarily help the model eliminate impossible movements.

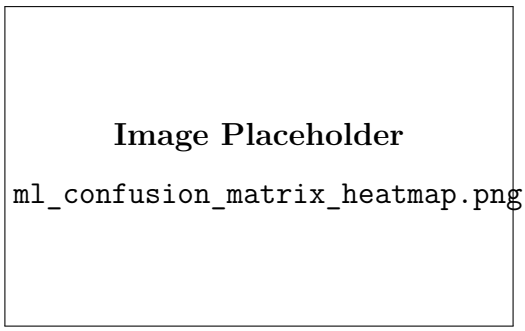
The SHAP beeswarm plot (Figure 7c) further explains how each individual feature affects the model's prediction results, and its reflected insights align consistently with the feature importance analysis. `prev1` exhibits the widest SHAP value distribution, exerting the strongest impact on prediction, whereas relative positions display narrower SHAP distributions, showing that they need to work synergistically with history and obstacle information and can rarely dominate predictions alone.

The ablation study (Figure 7d) confirms the importance of rationally combining all feature groups. The full model achieved the highest balanced accuracy, reaching 0.7649. When historical features were removed, performance dropped sharply to 0.5712, illustrating that the previous movement direction is absolutely indispensable. However, the model retaining only historical features (`history_only`) reached 0.6143, outperforming both the position-only and wall-only models. This indicates that movement history is the strongest single feature group, yet it remains insufficient to match the performance of the full model.

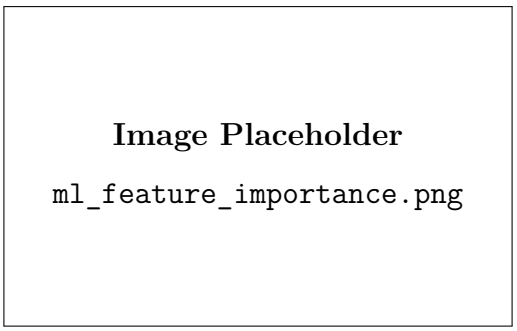
5 Conclusion

5.1 Achievements

The project successfully implemented Flask game backend that support an in-memory multi-game session architecture utilizing unique `game_id` tracking. It features two distinct interactive gameplay modes: an Escape mode where the player evades various AI algorithms, and a Chase mode where the player actively hunt a BFS-controlled human. For map creation, a C++ Genetic Algorithm was integrated to procedurally generate 30 x 30 wrap-around maze, optimizing fitness functions based on four main factors: reachability, wall ratio, junction complexity, and path distance. The system deployed a robust suite of pathfinding algorithms, including A* for multi-monster tracking, a Greedy controller, a Minimax controller utilizing Alpha-Beta pruning, and Simulated Annealing with lightweight game-play heuristics. On the frontend, a custom HTML5 Canvas-based UI featuring retro pixel-art aesthetics, dynamic keyframe animations, and an interactive HUD is designed to successfully bridge the visual presentation with the backend RESTful APIs. Furthermore, a automated compilation pipeline is established within the Flask backend to dynamically detect compilers (such as g++, clang++, or c1) and build

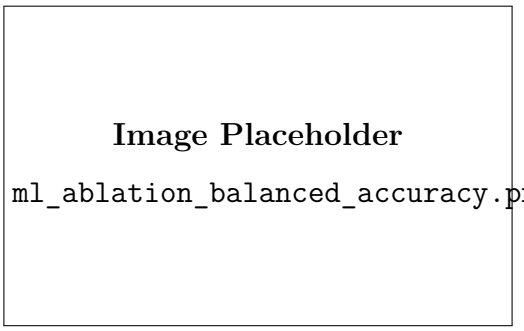


(a) Direction Prediction Confusion Matrix



(b) Random Forest Feature Importance

(c) SHAP Beeswarm Plot



(d) Ablation Study - Balanced Accuracy

Figure 7: Machine learning model evaluation

necessary C++ executables on demand without crash the web application.

5.2 Limitations

Despite these achievements, several algorithmic and computational constraints remain. Simulated Annealing (SA) exhibit a high degree of behavioral randomness on real-time chase scenario (achieving only a 83.3% capture rate) and suffer from high latency, with decision times reaching 9.99ms. While Minimax offers the highest capture efficiency (averaging 20.5 steps), its lookahead search results in a decision overhead of 0.62ms—significantly higher than A* (0.0511ms)—posing scalability bottlenecks for more complex mazes. Furthermore, deterministic algorithms like A* and Greedy lack the evolutionary ability to dynamically learn from and adapt to variable human escape strategies. The current predictive modeling also lack spatiotemporal dynamic feature fusion. It relies heavily on absolute positions (e.g., `human_col`, `human_row`) and brief step historie (e.g., `prev_down_1`), failing to model the player's long-term global strategic intentions. Additionally, environmental feature representation is overly restricted to immediate local wall constraints (`wall_up`, `wall_down`), completely missing macro-topological influences such as distant dead-ends and bifurcations. Finally, the integration of C++ algorithms currently relies on reading and writing to a temporary physical text file (`map/generated_map.txt`), introducing disk I/O bottlenecks during active web play. Additionally, the procedural map generation via Genetic Algorithms is computationally demanding and block the server terminal temporarily, causing delays during session initialization.

5.3 Future Work

Future work will be focused on advanced machine learning integration by enhancing the feature engineering pipeline to capture macro-topological maze structures (e.g., dead-ends, chokepoints) and temporal sequence data. Introducing Reinforcement Learning (such as Q-Learning) can empower the monster AI to dynamically adapt to player strategies rather than relying on static heuristics. For system and architecture optimization, the execution of C++ algorithm will transition from disk-based file I/O to direct memory binding or standard stream to eliminate temporary file dependencies during active gameplay. Moreover, upgrading the HTTP polling mechanism to WebSockets would enable seamless, realtime bidirectional communication and pave the way for PvP multiplayer. Finally, game rules will be expanded to support maze layouts with defined exit tiles, providing a definitive victory condition for the human player beyond simply maximizing survival duration. Furthermore, the Genetic Algorithm can be optimized to significantly reduce server-side processing times for initial maze generation.

References

...